

Documentación de Arquitectura - Sistema EcoWatch

Índice

1. [Visión General](#)
 2. [Arquitectura del Sistema](#)
 3. [Decisiones de Diseño](#)
 4. [Componentes Principales](#)
 5. [Flujo de Datos](#)
 6. [Modelo de Base de Datos](#)
 7. [Patrones Arquitectónicos](#)
-

Visión General

El Sistema EcoWatch implementa una **arquitectura hexagonal (Ports & Adapters)** con **separación por capas** para garantizar:

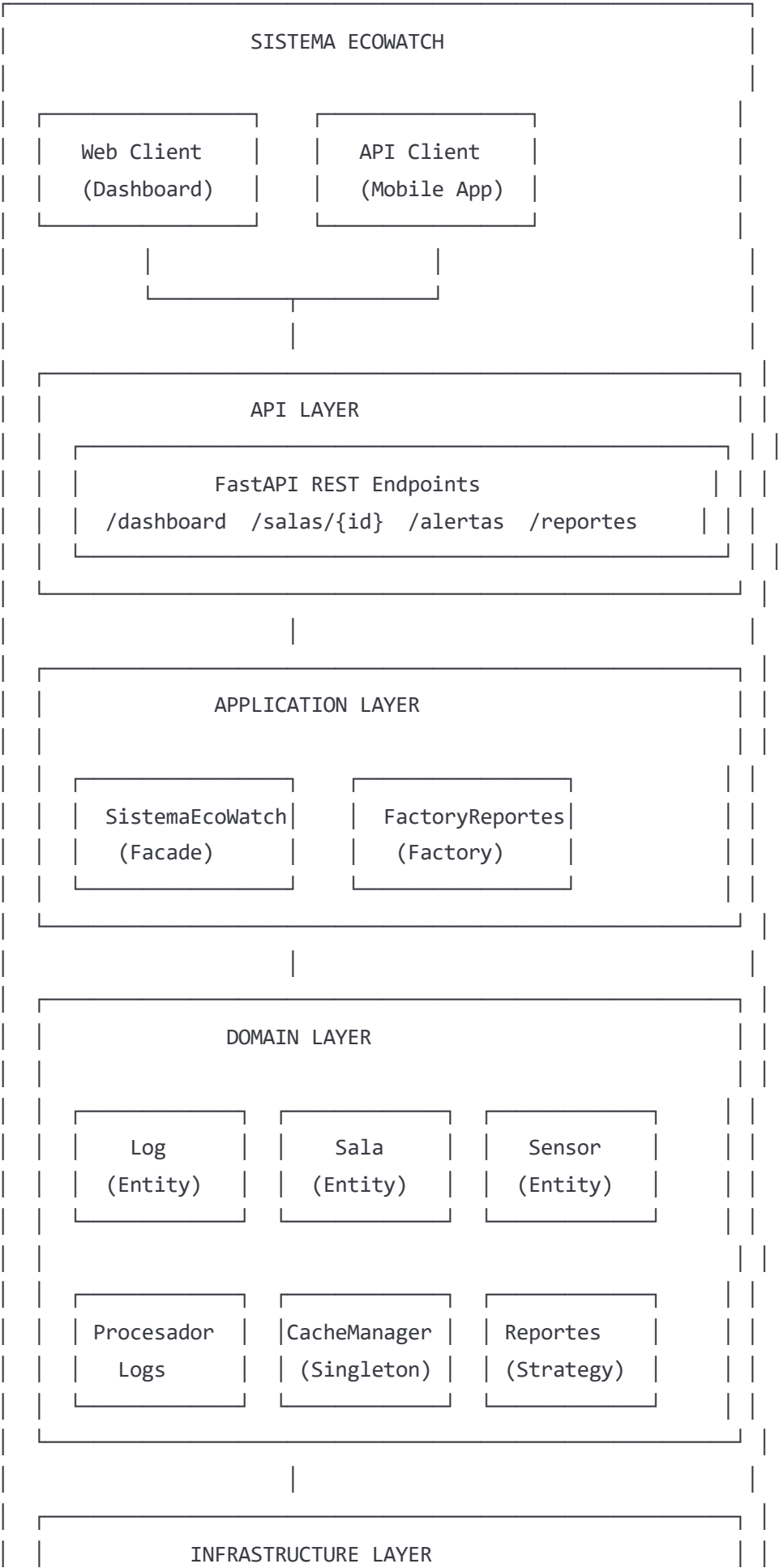
- **Mantenibilidad:** Código modular y bien organizado
- **Extensibilidad:** Fácil agregar nuevas funcionalidades
- **Testabilidad:** Componentes desacoplados y testeable por separado
- **Escalabilidad:** Preparado para crecimiento horizontal y vertical

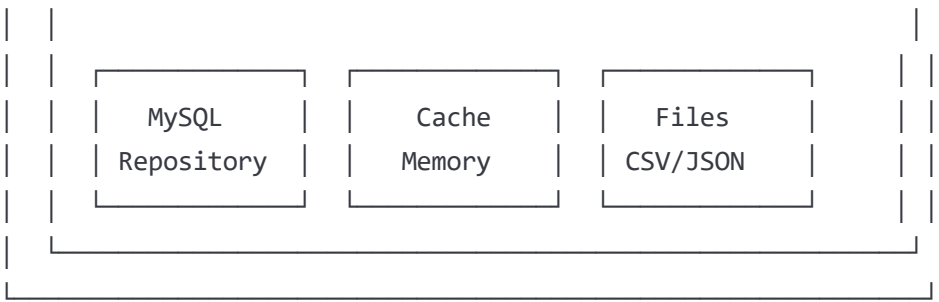
Principios Arquitectónicos

1. **Separation of Concerns:** Cada módulo tiene una responsabilidad específica
 2. **Dependency Inversion:** Dependencias apuntan hacia abstracciones
 3. **Open/Closed Principle:** Abierto para extensión, cerrado para modificación
 4. **Single Responsibility:** Cada clase tiene una única razón para cambiar
-

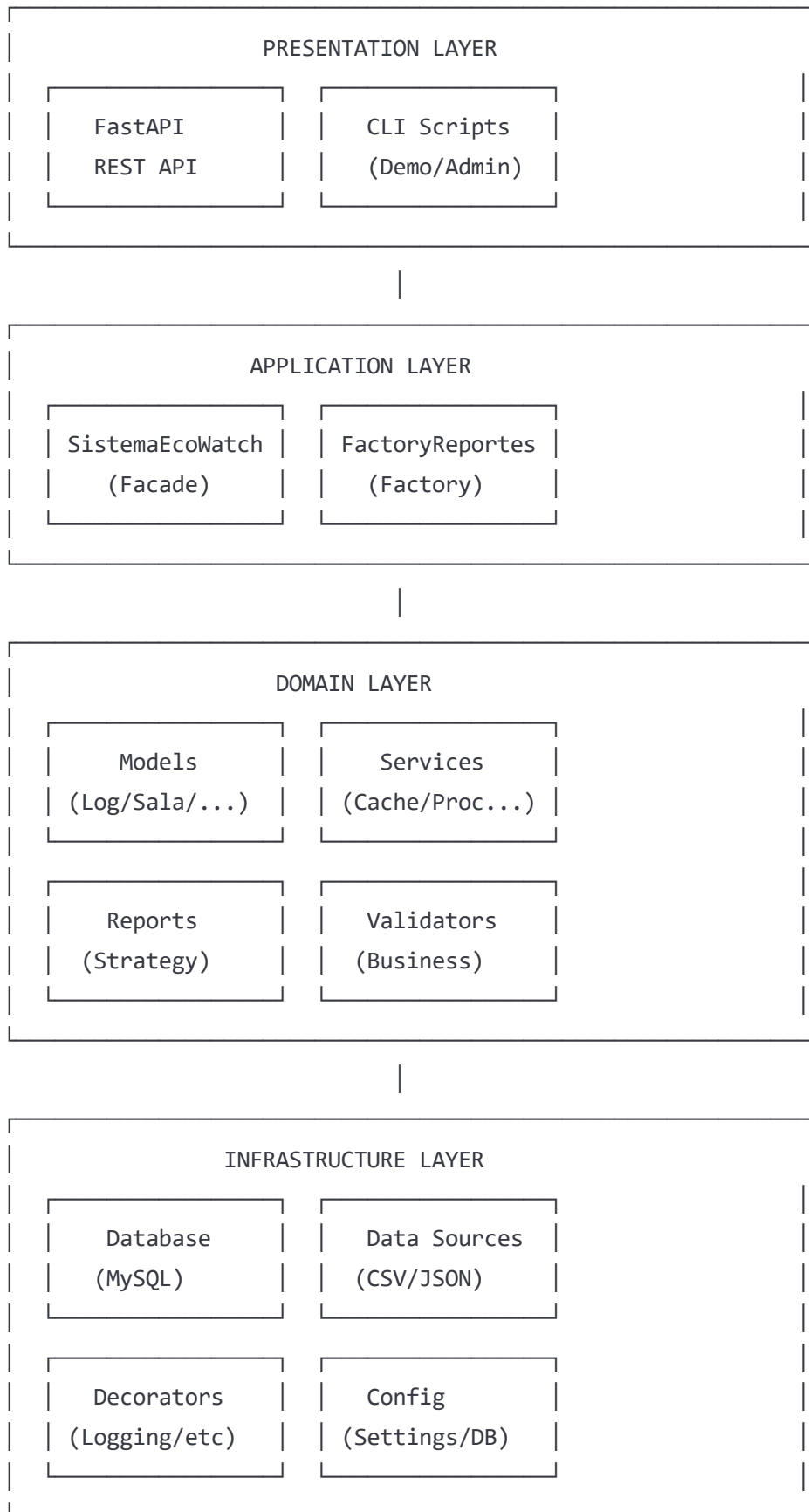
Arquitectura del Sistema

Diagrama de Arquitectura de Alto Nivel





Arquitectura por Capas



1. Arquitectura Hexagonal vs Layered

Decisión: Implementar arquitectura hexagonal con elementos de arquitectura por capas

Justificación:

- **Flexibilidad:** Permite cambiar adaptadores (DB, API, UI) sin afectar lógica de negocio
- **Testabilidad:** Core de negocio independiente de infraestructura
- **Evolución:** Facilita migración futura a microservicios

Alternativas consideradas:

- **Arquitectura en capas pura:** Descartada por alto acoplamiento
- **Microservicios:** Excesiva para MVP, pero arquitectura facilita migración futura

2. Python como Lenguaje Principal

Decisión: Python 3.8+ como lenguaje principal del sistema

Justificación:

- **Ecosistema:** Librerías maduras para análisis de datos (pandas, numpy)
- **Productividad:** Desarrollo rápido con sintaxis clara
- **Comunidad:** Gran comunidad y documentación
- **Machine Learning:** Preparado para futuras extensiones con ML/AI

Trade-offs aceptados:

- **Performance:** Aceptable para volúmenes actuales, mitigable con Cython/PyPy
- **Type Safety:** Compensado con type hints y validación en runtime

3. MySQL como Base de Datos

Decisión: MySQL 8.0+ como SGBD principal

Justificación:

- **ACID:** Transacciones confiables para datos críticos
- **Performance:** Excelente para cargas OLTP
- **Índices:** Soporte robusto para optimización de consultas
- **Ecosistema:** Herramientas maduras de administración y monitoreo

Alternativas consideradas:

- **PostgreSQL:** Descartado por menor familiaridad del equipo
- **NoSQL:** Inapropiado para datos estructurados con relaciones

4. Caché en Memoria vs Distribuido

Decisión: Caché en memoria para MVP, preparado para Redis

Justificación:

- **Latencia:** Menor latencia para consultas frecuentes
- **Simplicidad:** Menos componentes en la infraestructura inicial
- **Migración:** Diseño permite migración transparente a Redis

Plan de evolución:

```
python

# Actual: Caché en memoria
cache_manager = CacheTemporalManager()

# Futuro: Redis distribuido
cache_manager = RedisCacheManager(redis_client)
```

Componentes Principales

1. Modelos del Dominio (`models/`)

Responsabilidad: Entidades de negocio con lógica y validaciones

python

```
@dataclass
class Log:
    """Entidad principal del dominio"""
    timestamp: datetime
    sala: str
    estado: EstadoLog
    temperatura: float
    humedad: float
    co2: int
    mensaje: str

    def __post_init__(self):
        self.is_critical = self._calcular_criticidad()
```

Características:

- **Inmutabilidad:** Datos críticos protegidos
- **Validación automática:** Post-init hooks para verificaciones
- **Lógica de negocio:** Métodos para cálculos específicos del dominio

2. Servicios de Aplicación (services/)

Responsabilidad: Orquestación de casos de uso de negocio

python

```
class ProcesadorLogs:
    """Orquesta el procesamiento de logs"""

    @log_operation
    @benchmark
    def procesar_fuente(self, fuente: FuenteDatos) -> int:
        # Lógica de orquestación
```

Características:

- **Decoradores:** Funcionalidades transversales (logging, benchmarking)
- **Dependency Injection:** Recibe dependencias por constructor
- **Error Handling:** Manejo robusto de excepciones

3. Sistema de Reportes (`reports/`)

Responsabilidad: Generación flexible de reportes ejecutivos

```
python

# Factory Pattern
reporte = FactoryReportes.crear_reporte(TipoReporte.ESTADO_POR_SALA)

# Strategy Pattern
reporte_tendencias = ReporteTendencias(AnalisisTendencias())
```

Características:

- **Factory Pattern:** Creación desacoplada de reportes
- **Strategy Pattern:** Algoritmos intercambiables de análisis
- **Template Method:** Flujo común de generación

4. Acceso a Datos (`database/`)

Responsabilidad: Persistencia y consulta de datos

```
python

# Repository Pattern
logs = LogRepository.get_recent_logs(minutes=5)

# Connection Pool
with DatabaseConnection.get_connection() as conn:
    # Operaciones de BD
```

Características:

- **Repository Pattern:** Abstrae el acceso a datos
- **Connection Pooling:** Reutilización eficiente de conexiones
- **Migration Management:** Versionado del esquema

Flujo de Datos

1. Flujo de Ingesta de Datos

CSV/JSON/API → DataSource → Validator → Processor → Database

↓

Cache ← Alerts ← Business Rules

Descripción detallada:

1. **Fuente de datos** lee desde archivo/API/BD
2. **Validador** verifica estructura y tipos
3. **Procesador** aplica reglas de negocio
4. **Base de datos** persiste los datos
5. **Caché** almacena datos recientes para consultas rápidas
6. **Sistema de alertas** evalúa condiciones críticas

2. Flujo de Generación de Reportes

Request → Factory → Strategy → Data Access → Analysis → Response

↓

Cache Check → Repository → Business Logic → Template → Export

Descripción detallada:

1. **Request** especifica tipo de reporte y parámetros
2. **Factory** crea la instancia de reporte apropiada
3. **Strategy** selecciona algoritmo de análisis
4. **Data Access** obtiene datos (caché o BD)
5. **Business Logic** aplica lógica específica del reporte
6. **Template** formatea la salida
7. **Export** genera archivo en formato solicitado

3. Flujo de Consulta en Tiempo Real

API Request → Facade → Cache Manager → Index Lookup → Response

↓

Authentication → Authorization → Rate Limiting → Logging

Esquema Relacional

sql

-- Tabla principal de Logs

```
CREATE TABLE logs (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    timestamp DATETIME NOT NULL,  
    sala VARCHAR(50) NOT NULL,  
    estado ENUM('INFO', 'WARNING', 'ERROR') NOT NULL,  
    temperatura DECIMAL(5,2) NOT NULL,  
    humedad DECIMAL(5,2) NOT NULL,  
    co2 INT NOT NULL,  
    mensaje TEXT,  
    processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    is_critical BOOLEAN DEFAULT FALSE,  
  
    -- Índices para optimización  
    INDEX idx_timestamp (timestamp),  
    INDEX idx_sala (sala),  
    INDEX idx_critical (is_critical),  
    INDEX idx_sala_timestamp (sala, timestamp)  
);
```

-- Tabla de salas

```
CREATE TABLE salas (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(50) UNIQUE NOT NULL,  
    ubicacion VARCHAR(100),  
    capacidad_personas INT DEFAULT 0,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

-- Tabla de sensores

```
CREATE TABLE sensores (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    id_sensor VARCHAR(50) UNIQUE NOT NULL,  
    sala_id INT,  
    tipo ENUM('temperatura', 'humedad', 'co2', 'multi') NOT NULL,  
    activo BOOLEAN DEFAULT TRUE,  
    ultima_lectura DATETIME,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    FOREIGN KEY (sala_id) REFERENCES salas(id)  
);
```

Justificación del Diseño

1. **Índices compuestos:** `(sala, timestamp)` para consultas frecuentes por sala y período
2. **Campos calculados:** `is_critical` desnormalizado para consultas rápidas de alertas
3. **Enums:** Estados y tipos restringidos para integridad de datos
4. **Timestamps:** `processed_at` para auditoría y `updated_at` para control de cambios

Estrategia de Escalamiento

```
sql

-- Particionamiento por fecha (futuro)
CREATE TABLE logs_2025_01 PARTITION OF logs
FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');

-- Índices especializados para consultas específicas
CREATE INDEX idx_logs_critical_recent
ON logs (is_critical, processed_at)
WHERE is_critical = TRUE;
```

Patrones Arquitectónicos

1. Hexagonal Architecture (Ports & Adapters)

Implementación:

```
python

# Port (interfaz)
class FuenteDatos(Protocol):
    def leer_logs(self) -> List[Dict[str, Any]]: ...

# Adapters (implementaciones)
class FuenteCSV(FuenteDatos): ...
class FuenteJSON(FuenteDatos): ...
class FuenteAPI(FuenteDatos): ...
```

Beneficios:

- Core de negocio independiente de infraestructura
- Fácil testing con mocks
- Flexibilidad para cambiar adaptadores

2. Layered Architecture

Capas definidas:

- **Presentation:** FastAPI, CLI
- **Application:** SistemaEcoWatch (Facade)
- **Domain:** Models, Services, Reports
- **Infrastructure:** Database, Cache, Files

Reglas de dependencia:

- Capas superiores pueden depender de inferiores
- Capas inferiores NO dependen de superiores
- Inyección de dependencias para inversión de control

3. Domain-Driven Design (DDD) Elements

Entidades:

```
python

@dataclass
class Log: # Entity with identity
    id: Optional[int] = None
```

Value Objects:

```
python

@dataclass(frozen=True)
class CondicionAmbiental: # Immutable value object
    temperatura: float
    humedad: float
    co2: int
```

Services:

```
python

class ProcesadorLogs: # Domain service
    def procesar_log_individual(self, log_data): ...
```

Repositories:

python

```
class LogRepository: # Infrastructure service
    def get_recent_logs(self, minutes): ...
```

Métricas de Calidad Arquitectónica

1. Acoplamiento

- **Bajo acoplamiento:** Módulos independientes comunicándose por interfaces
- **Dependency Injection:** Dependencias inyectadas, no hard-coded
- **Event-driven:** Comunicación asíncrona donde apropiado

2. Cohesión

- **Alta cohesión:** Cada módulo tiene responsabilidad única y bien definida
- **SRP:** Cada clase tiene una única razón para cambiar
- **Functional cohesion:** Elementos trabajan juntos hacia un objetivo común

3. Complejidad

- **Complejidad ciclomática:** Mantened baja con funciones pequeñas
- **Depth of inheritance:** Preferencia por composición sobre herencia
- **Fan-in/Fan-out:** Balanceado para evitar cuellos de botella

4. Mantenibilidad

- **Código autodocumentado:** Nombres descriptivos y funciones pequeñas
 - **Separation of concerns:** Cada archivo/clase con propósito claro
 - **DRY principle:** Sin duplicación de lógica
 - **YAGNI:** Implementación solo de lo necesario
-

Evolución Futura

Migración a Microservicios

Preparación actual:

- Módulos bien definidos y desacoplados

- Interfaces claras entre componentes
- Base de datos por contexto (logs, reportes, configuración)

Roadmap de migración:

1. **Paso 1:** Extraer servicio de reportes
2. **Paso 2:** Separar ingesta de datos
3. **Paso 3:** Microservicio de alertas en tiempo real
4. **Paso 4:** API Gateway y service mesh

Escalamiento Horizontal

Componentes stateless: Preparados para múltiples instancias **Cache distribuido:** Migración transparente a Redis **Database sharding:** Particionamiento por sala o fecha **Load balancing:** API preparada para balanceadores

Integración de Machine Learning

Arquitectura preparada:

```
python

# Future ML integration
class AnalisisML(EstrategiaAnalisis):
    def __init__(self, modelo_path):
        self.model = load_model(modelo_path)

    def analizar(self, logs):
        features = self.extract_features(logs)
        return self.model.predict(features)
```

Casos de uso:

- Predicción de fallos de equipos
- Detección de anomalías
- Optimización automática de HVAC
- Forecasting de demanda energética